

On sait que le programme termine, car la boucle `for` itère un nombre fini de fois (la taille des tableaux).

Correction: en montrant l'invariant : "après i tours de boucle, $res \equiv n_1 + n_2[10^i]$ " (ie. les i premiers chiffres de la somme sont corrects). On peut faire une preuve par récurrence sur i .

Complexité en temps: autant d'additions élémentaires (ie. de chiffres) que de chiffres dans les nombres. Donc **linéaire EN la taille des entrées** (préciser linéaire en quoi est important, éviter les sous-entendus).

Donc ici, dire que " n_1 et n_2 sont de taille au plus l " signifie que $n_1, n_2 \in O(10^l)$, ou encore que $l = 1 + \lfloor \max(\log_{10}(n_1), \log_{10}(n_2)) \rfloor$.

 **Exemple :** Multiplication de deux entiers.

```

1 def multiplication_naive(nb1, nb2) :
2     # nb1 tableau de chiffres représentant un entier n1
3     # nb2 entier n2 représenté de manière usuelle (type int)
4     res = [0] * len(nb1) # tableau de 0 de longueur identique que nb1
5     for i in range(1, nb2+1) : # de i=1 à i=nb2, donc nb2 tours
6         res = addition(res, nb1)
7     return res

```

Correction: similaire à l'addition, en utilisant l'invariant "après l'étape i , $res \equiv n_1 \times i[10^l]$ " où l est la taille de n_1 .

Complexité en temps: n_2 additions, de grands entiers. Chaque addition est de coût linéaire en la taille du résultat, donc en $\log(n_1 \times n_2) = \log(n_1) + \log(n_2)$.

Complexité en $O(n_2 \times (\log(n_1 \cdot n_2)))$, soit $O(l \times 10^l)$ si n_1 et n_2 sont de taille l .

 **Remarque :** On manipulera toujours la partie entière supérieure de $\log_{10}(n)$ pour un entier n comme la "taille" de n .

sez z